

2026 编译实验设计文档

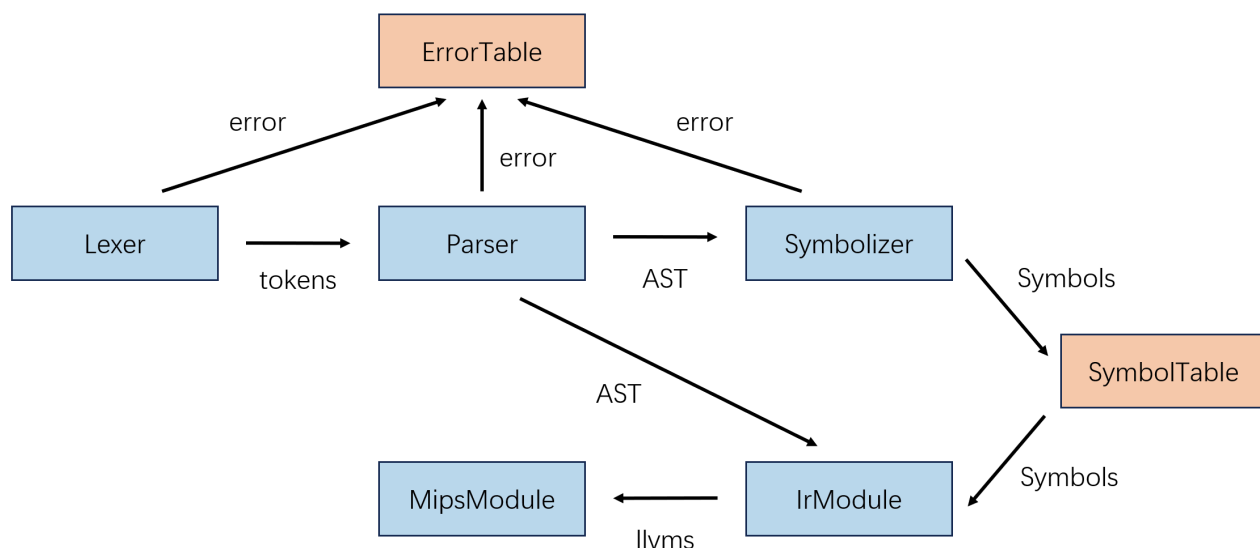
1. 参考编译器介绍

1.1 编译器总体结构

参考的编译器是课程组在课程文件中提供的编译器代码。该编译器是实现 SysY 语言到 MIPS 汇编语言转换的编译器，使用 java 语言实现。根据编译器的 7 个逻辑部分，可将编译器分为不同的模块，分别为前端的词法分析、语法分析，中端的语义分析、符号表管理、中间代码生成、代码优化，后端的目标代码生成，以及跨越前端和中端的异常处理。

1.2 编译器接口设计

编译器的接口设计如图：



1.3 编译器文件组织

编译器的文件组织如下：

```

src/
|- Compiler      主函数，编译器的入口
|- utils/        用于进行 I/O 、计算 Final Cycle 、设置运行参数与 debug
|- error/        用于管理词法分析、语法分析、语义分析中出现的异常
|- frontend/     编译器前端
|   |- FrontEnd  编译器前端入口
|   |- lexer/    用于进行词法分析
|   |- parser/   用于进行语法分析
|   \- ast/      用于存储语法分析生成的 AST
|- midend/       编译器中端
|   |- MidEnd    编译器中端入口
|   |- symbol/   用于存储生成的符号表
|   |- visit/    用于进行中间代码生成
|   \- llvm/     用于存储生成的中间代码
|- optimize/     用于进行各种优化
  \- backend/    编译器后端
      |- BackEnd  编译器后端入口
      |- mips/    用于存储生成的目标代码
      \- PeepHole 用于进行窥孔优化

```

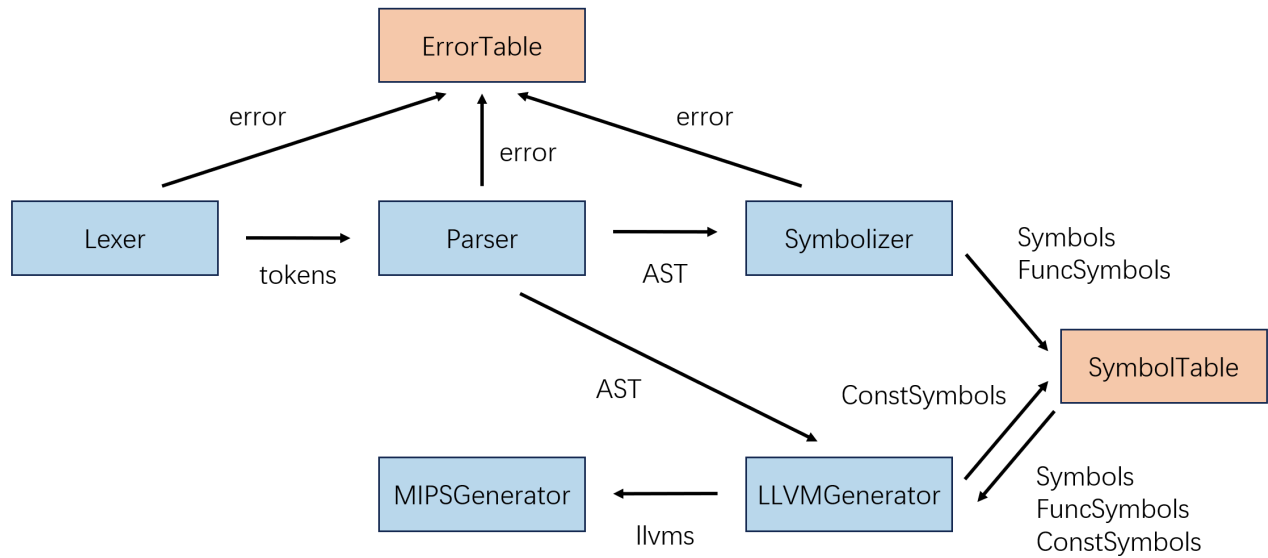
2. 编译器总体设计

2.1 编译器总体结构

本编译器是实现 SysY 语言到 MIPS 汇编语言转换的编译器，使用 java 语言实现。根据编译器的 7 个逻辑部分，可将编译器分为不同的模块，分别为前端的词法分析、语法分析，中端的语义分析、符号表管理、中间代码生成、代码优化，后端的代码生成，以及跨越前端和中端的异常处理。

2.2 编译器接口设计

编译器的接口设计如图：



2.3 编译器文件组织

编译器的文件组织如下：

```

src
|- Compiler          主函数，编译器的入口
|- error/            用于管理词法分析、语法分析、语义分析中出现的异常
|- lexer/            用于进行词法分析
|- parser/           用于进行语法分析
|- symbolizer/       用于进行语义分析和符号表生成
|- llvmgenerator/    用于进行中间代码生成
\ - mipsgenerator/   用于进行目标代码生成

```

3. 词法分析

3.1 设计部分

【词法分析算法设计】

词法分析主要分为两步：第一步是在整个字符串形式的文件中读取当前的 Token，此时 Token 为字符串形式；第二步则是将字符串形式的 Token 转换为 Token 类，并存入 tokens 数组中。

我们要识别的 Token 主要分为这几种类型：关键字、变量名、整数常量、字符常量，以及其它符号。此外，我们还需要在识别 Token 的过程中完成对两种注释（单行注释和多行注释）的清除。

在读取 Token 时，我们首先读取当前的字符，作为 Token 的第一个字符。如果当前的字符为下划线或英文大小写字母，则需要继续读取，直到后面的字符不再是下划线、数字或英文大小写字母，或文件结束。此时，若当前的字符串为保留的关键字，则输出关键字对应的 Token，否则识别为变量名，输出 IDENFR 类型的 Token。

如果当前的字符为数字，则需要继续读取，直到后面的字符不再是数字，或文件结束。此时识别为整数常量，输出 INTCON 类型的 Token。

如果当前的字符为双引号，则需要继续读取，直到读取到下一个双引号，或文件结束。此时识别为字符常量，输出 STRCON 类型的 Token。

如果当前的字符为其它字符，若该字符能够组成双字符的 Token，如 `!=`，则需要预读接下来的一个字符，判断当前的 Token 是单字符 Token 还是双字符 Token。

特别地，如果当前的字符为 `/`，除了表示除法外，还可能组成单行注释 `//` 和多行注释 `/*`。当确定为单行注释时，需要继续读取，直到读取到 `\n` 换行符，或文件结束；当确定为多行注释时，需要继续读取，直到读取到连续的 `*/`，或文件结束。此时不输出任何类型的 Token。

在识别 Token 的过程中，我们还会遇到不属于任何 Token 的符号，如空格和换行符 `\n`。当我们读到的字符串形式的 Token 不属于任何 Token 时，需要能够不输出任何类型的 Token，而非直接报错。

此外，为了能够在异常处理中实现对行号的识别，我们在 Token 类中，除了要保存 Token 的内容（字符串格式）和种类，还需要保存当前 Token 所在的行号。实现起来也比较简单，具体来说可以维护一个保存当前行号的变量，每当我们读到 `\n` 字符时，将该变量加 1 即可。

【词法分析异常处理】

词法分析的异常种类非常简单，大概就是让大家简单试试水。这里的异常一共分为两种，一种是将 `&&` 写成了 `&`，另一种是将 `||` 写成了 `|`。

显然，两种异常的实现方法如出一辙，这里以 `&&` 作为例子说明。可以参考之前双字符 Token 的识别方法，在读取 Token 时，若首字符为 `&`，则预读接下来的一个字符。若为 `&`，则正常识别为 `&&`；若不为 `&`，则报告异常。需要注意的是，为了接下来的语法和语义部分能够正常进行异常分析，即使出现了异常，我们仍然要将 `&&` 对应的 Token 加入 `tokens` 中。

【文件的输出】

输出功能依赖于 `java.nio.file` 中的 `Files.write()` 方法，我们使用 `Path` 类指定输出路径，并将需要输出的字符串传入，即可实现向文件输出的效果。

需要注意的是，这种输出方法不支持文件的追加写入，所以我们需要将所有要输出的内容全部写入同一个字符串中，最后再进行输出。这里使用 `StringBuilder` 类对象进行字符串的追加写入，防止每次对 `String` 类对象进行追加时，均需要创建新的对象：

```
public void printError() throws IOException {
    StringBuilder strb = new StringBuilder();
    for (Error error : errors) {
        strb.append(error.getLine());
        strb.append(" ");
        strb.append(error.getType());
        strb.append("\n");
    }
    Path path = Paths.get("error.txt");
    Files.write(path, strb.toString().getBytes());
}
```

3.2 实现部分

【实现词法分析器】

为了实现词法分析，我们首先创建枚举类 `TokenType`，在其中注册所有的 `Token` 类型：

```
public enum TokenType {
    IDENFR,
    INTCON,
    STRCON,
    .....
}
```

接下来实现 `Token` 类，记录每个 `Token` 的类型、内容和行号：

```
public class Token {
    private TokenType tokenType;
    private String content;
    private int line;
}
```

最后实现 `Lexer` 类，对输入的 `code` 进行分析，输出 `tokens`：

```

public class Lexer {
    private final String code;
    private final int size;
    private int nowat;
    private int nowline;
    private ArrayList<Token> tokens;
    private ErrorList errorList;
}

```

文件结构如下：

```

src/
|- Compiler
|- lexer/
|   |- TokenType
|   |- Token
|   \- Lexer
|- ...

```

【实现异常处理】

为了实现异常处理，我们首先为每条异常创建类 `Error` ，记录异常的行号和类型：

```

// error/Error
public class Error {
    private int line;
    private char type;
}

```

在此之上，再创建存储所有异常的类 `ErrorList` ，用于将所有异常输出至文件中：

```

// error/ErrorList
public class ErrorList {
    private ArrayList<Error> errors;
}

```

文件结构如下：

```
src/  
|- Compiler  
|- error/  
|   |- Error  
|   \- ErrorList  
|- ...
```

【实现主函数】

秉承着主函数只进行简单调用的原则，主函数只负责文件的读取，`errorList` 和 `lexer` 的创建，启动词法分析，以及启动相应的输出。

在文件读取部分，我们采用 `java.nio.file` 中的 `Files.readAllBytes()` 方法，配合 `Path` 类的文件路径，即可将整个 `testfile` 文件输入至字符串 `code` 中。

接下来创建 `errorList`，并在创建 `lexer` 的时候传入其中。当 `lexer` 检测到异常时，可以直接创建相应的 `error`，写入 `errorList` 中。

随后创建 `lexer`，对 `code` 进行词法分析，将分析的结果输出至 `tokens` 中，用于接下来的语法分析。

最后根据 `errorList` 是否为空，判断应输出 `error.txt` 还是 `lexer.txt`。主函数代码如下：

```
// Compiler  
public class Compiler {  
    public static void main(String[] args) throws IOException {  
        Path path = Paths.get("testfile.txt");  
        String code = new String(Files.readAllBytes(path));  
  
        ErrorList errorList = new ErrorList();  
  
        Lexer lexer = new Lexer(code, errorList);  
        ArrayList<Token> tokens = lexer.lex();  
        if (errorList.hasError()) {  
            errorList.printError();  
        }  
        else {  
            lexer.printToken();  
        }  
    }  
}
```

4. 语法分析

4.1 设计部分

【语法分析算法设计】

虽然我们使用的文法不是 LL(1) 文法，但是我们可以通过一些手段，使得我们依然能够使用递归子程序法对其进行分析。对于文法中的左递归问题，我们可以对文法进行适当改写；对于文法中的回溯问题，我们可以使用无限预读的方式进行解决。

首先是文法中的左递归问题。在我们的文法中，乘除模表达式、加减表达式、关系表达式、相等性表达式、逻辑与表达式、逻辑或表达式均存在左递归问题：

```
乘除模表达式 MulExp → UnaryExp | MulExp ('*' | '/' | '%') UnaryExp
加减表达式 AddExp → MulExp | AddExp ('+' | '-') MulExp
关系表达式 RelExp → AddExp | RelExp ('<' | '>' | '<=' | '>=') AddExp
相等性表达式 EqExp → RelExp | EqExp ('==' | '!=') RelExp
逻辑与表达式 LAndExp → EqExp | LAndExp '&&' EqExp
逻辑或表达式 LOrExp → LAndExp | LOrExp '||' LAndExp
```

针对此种情况，我们采用扩充的 BNF 范式改写文法：

```
乘除模表达式 MulExp → UnaryExp { ('*' | '/' | '%') UnaryExp }
加减表达式 AddExp → MulExp { ('+' | '-') MulExp }
关系表达式 RelExp → AddExp { ('<' | '>' | '<=' | '>=') AddExp }
相等性表达式 EqExp → RelExp { ('==' | '!=') RelExp }
逻辑与表达式 LAndExp → EqExp { '&&' EqExp }
逻辑或表达式 LOrExp → LAndExp { '||' LAndExp }
```

文法中还会出现回溯问题，有些回溯问题可以通过有限次的预读解决，例如编译单元 CompUnit 的文法：

```
编译单元 CompUnit → {Decl} {FuncDef} MainFuncDef
```

其中 Decl 和 FuncDef 均可以以 int 开头，即二者的 FIRST 集有交集，无法通过预读一个 Token 的方式将二者区分开。不过，当二者都以 int 开头时，Decl 的前三个 Token 为 int IDENFR = int IDENFR , int IDENFR [int IDENFR ; 其中的一个，而 FuncDef 的前三个 Token 为 int IDENFR (, 所以我们通过预读三个 Token , 就能够解决该文法中的回溯问题。

然而，也有一些回溯问题是无法通过预读有限个 Token 解决的，例如语句 Stmt 的文法：


```

语句 Stmt → LVal '=' Exp ';'
| [Exp] ';'
| Block
| 'if' '(' Cond ')' Stmt [ 'else' Stmt ]
| 'for' '(' [ForStmt] ';' [Cond] ';' [ForStmt] ')' Stmt
| 'break' ';'
| 'continue' ';'
| 'return' [Exp] ';'
| 'printf'('StringConst {' ','Exp'})'';'

```

其中， $\text{Stmt} \rightarrow \text{LVal} \text{'=' Exp ';'}$ | $[\text{Exp}] \text{';'}$ 就会出现上述问题，因为 Exp 经过多步推导恰好可以推导为 LVal ，而 LVal 含有的 Token 数又可以无限多，所以无法通过预读有限个 Token 的方式解决。

不幸中的万幸， LVal 中并不可以出现 $=$ 或者 $;$ ，所以我们可以通过无限预读的方式，不停读取接下来的 Token。如果先遇到了 $=$ ，那么就进入 $\text{Stmt} \rightarrow \text{LVal} \text{'=' Exp ';'}$ 分支；如果先遇到了 $;$ ，那么就进入 $\text{Stmt} \rightarrow [\text{Exp}] \text{';'}$ 分支。

事实上，并不是所有的非二义性文法的回溯问题都能够通过无限预读的方式得到确定的结果，例如上面的例子中，如果 LVal 中可能会出现 $=$ 和 $;$ ，那么这种无限预读的方法就寄了，还是要通过回溯的方式解决问题。（希望期中期末不要考这种变态的文法吧）

解决了左递归和回溯问题，我们就可以正常进行递归子程序法的分析了！至于递归子程序法的具体细节，在这里就不详述了。

【语法分析异常处理】

语法分析中一共会出现三种异常，分别是缺少分号、缺少右小括号、缺少右中括号。

这些异常看似比较简单，一种想法是，只需要在分析这些符号的时候观察它们是否存在就可以了，然而这种想法还是太天真了！真正存在隐患的是，这些符号均有可能出现在文法的末尾，发生缺失时可能会改变文法的 FOLLOW 集，进而是原本不存在回溯问题的文法出现回溯问题，甚至导致文法退化为二义性文法！

例如我们观察下面这些例子，它们都是存在二义性的文法：

```

5 -3;    // 5 - 3; 还是 5; -3;
func(2 + 3;    // func(2) + 3; 还是 func(2 + 3);
arr[2 + 3;    // arr[2] + 3; 还是 arr[2 + 3];

```

可以发现，缺失 $;$ $)$ $]$ 均可能会引发二义性问题，那么对此我们应该如何解决呢？实际上，对于可能出现的异常，我们应该遵循“宁可信其无，不可信其有”的规则。即只要当前的 Token 序列能够被正常分

析，我们就不考虑可能会出现异常的情况；只有当前的 Token 序列无法再分析下去的时候，我们才会考虑是否出现了异常。

例如上面例子中的 `func(2 + 3;`，当我们分析完 `func(2` 之后，读到 `+` 时，此时是可以正常将 `+` 识别为加减表达式中的 `+`，所以不认为此处有异常，正常进行分析；当我们分析完 `func(2 + 3`，读到 `;` 时，此时没有任何相应的文法能够支持此处的 `;`，所以只能认为出现了缺少右小括号的异常。在上面的例子中，三条语句实际上应该被分析为 `5 - 3;` `func(2 + 3);` 和 `arr[2 + 3];`。

此外，用语法分析开始，我们加入 `errorTable` 中的异常顺序会出现和输出要求的顺序不一样的情况，所以在输出之前，我们还需要根据每个异常所在的行号，对所有异常重新排序再进行输出。

4.2 实现部分

【实现语法分析器】

为了实现语法分析，我们创建语法分析器 `Parser` 类，将词法分析得到的 Token 序列 `tokens`，以及异常表 `errorList` 传入其中，即可构建出与文法对应的 AST：

```
// parser/Parser
public class Parser {
    private final ArrayList<Token> tokens;
    private final int size;
    private int nowat;
    private CompUnit compUnit;
    private ErrorList errorList;
}
```

在 `parser` 文件夹中，我们为每一个文法的非终结符构造相应的类，文件结构如下：

```
src/
|- Compiler
|- parser/
|   |- Parser
|   |- CompUnit
|   |- type/
|   |   |- BType
|   |   \- FuncType
|   |- declaration/
|   |   |- ConstDecl
|   |   |- ConstDef
|   |   |- ConstInitVal
|   |   |- Decl
|   |   |- InitVal
|   |   |- VarDecl
|   |   \- VarDef
|   |- function/
|   |   |- FuncDef
|   |   |- FuncFParam
|   |   \- FuncFParams
|   |- block/
|   |   |- Block
|   |   |- BlockItem
|   |   |- Cond
|   |   |- ForStmt
|   |   |- MainFuncDef
|   |   |- Stmt
|   |   |- StmtAssign
|   |   |- StmtBlock
|   |   |- StmtBreak
|   |   |- StmtContinue
|   |   |- StmtExp
|   |   |- StmtFor
|   |   |- StmtIf
|   |   |- StmtPrint
|   |   \- StmtReturn
|   \- expression/
|       |- AddExp
|       |- ConstExp
|       |- EqExp
|       |- Exp
|       |- FuncRParams
|       \- LandExp
```

```
|      |- LOrExp
|      |- LVal
|      |- MulExp
|      |- Number
|      |- PrimaryExp
|      |- RelExp
|      |- UnaryExp
|      \- UnaryOp
|- ...
```

5. 语义分析

5.1 设计部分

【符号表设计】

在语义分析阶段，我们需要为 AST 建立符号表。符号表中的每个 Symbol 都由三个部分组成：名称、类型、作用域。通过遍历 AST，我们能够非常轻松地找到所有变量的定义点，而获得每个变量的作用域则略显复杂，下面将着重介绍如何计算变量的作用域。

为了计算变量的作用域，我们设置一个 Scope 类，同时维护两个变量：一是当前最新作用域的编号 nowMaxScope，用于为新的作用域分配编号；二是当前作用域形成的栈 scopeStack，用于在退出当前作用域时能够回到上一级作用域。

在遍历 AST 时，当我们进入一个 Block，就需要为进入的新作用域分配编号，此时 nowMaxScope++，并将其压入栈中；当我们离开 Block 时，就需要将栈顶的作用域弹出（注意这里并不会 nowMaxScope--）。

需要注意的是，在函数定义时，会出现这样的问题：函数的参数实际上属于函数内部的作用域，但因为其不在 Block 中会出现错误。为了打上这个补丁，我们在定义函数参数前可以临时执行 nowMaxScope++，并将其压入栈中；在函数参数定义结束后，我们再将刚才临时的作用域从栈中弹出，并执行 nowMaxScope--，假装无事发生。这样，我们就实现了符号表的设计。

【语义分析异常处理】

在之前的环节中，构建 Token 序列和 AST 是词法分析和语法分析的重点，而异常处理只能算是开胃小菜。而语义分析恰恰相反，构建符号表其实非常简单，真正的重头戏是如何处理各种各样的奇葩异常。

首先是变量名重定义和未定义异常。二者看似相似，实则大不相同。首先，重定义异常需要在变量的定义点检测，而未定义异常需要在变量的使用点检测；其次，重定义异常只需要检测当前作用域内是否有重名的变量，而未定义异常则需要在整个作用域栈内检测是否定义过该变量。

在实现上，我们需要获取待检测变量的 Token，根据 Token 的名字和当前的作用域（或作用域栈）在符号表中进行查找。若出现异常，直接使用待检测变量的 Token 的所在行数作为异常的所在行数。

需要特别注意的是，对于 `getint()` 这个特殊的函数，我们需要进行特殊处理。因为 `getint()` 函数不需要定义即可直接使用，所以当 Token 名为 `getint` 时，如果是检测重定义，则直接报告异常；如果是检测未定义，则不报告异常。

接下来是函数的参数类型和个数不匹配。由于我们在符号表中并没有存储函数参数相关的信息，所以当程序调用函数时，我们无法在符号表中检查参数信息是否匹配。为此，我们可以为函数再建立一张符号表，使用 `FuncSymbol` 存储函数相关信息，包括 `Symbol` 中有的名称、类型、作用域，以及 `Symbol` 中没有的参数信息，使用 `ArrayList` 存储每个参数的类型，`ArrayList` 的大小即为参数个数。

在建立函数符号表之后，我们就可以使用函数名的 Token，根据函数的名字在符号表中查找到函数的参数信息进行比较。若出现异常，即可使用函数名的 Token 的所在行数作为异常的所在行数。

再接下来则是无返回值的函数存在不匹配的 `return` 语句，以及有返回值的函数缺少 `return` 语句。可以说这两个异常属实是有够逆天，文字游戏这一块。

无返回值的函数存在不匹配的 `return` 语句，不是说 `void` 类型的函数只要出现 `return` 语句就抛出异常，实际上在 `void` 类型的函数中使用 `return`；直接返回是可以的，而在 `return` 后面加上返回值是不可以的。所以我们的判断过程应该是这样的：

我们在执行到 `StmtReturn` 的时候进行判断，需要向判断函数传入 `return` 的 Token，以及一个布尔类型变量，表示当前的 `return` 语句是否有返回值。接下来，在判断函数中，我们直接从 `FuncSymbolTable` 中读取位于栈顶的 `FuncSymbol`，这就是当前正在执行的函数，如果这个函数是 `void` 类型，且 `return` 语句有返回值，我们才会抛出异常，否则不认为存在异常。

有返回值的函数缺少 `return` 语句，指的单纯就是函数的最后一行，即最后一个 `BlockItem` 不是 `StmtReturn`，此外的所有情况，哪怕是最后一行是 `return`；，也不认为是存在异常。于是我们只需要在函数定义的时候进行判断，将函数的右括号的 Token，以及当前函数中最后一条 `BlockItem` 传入函数中，由判断函数判断其是不是 `StmtReturn` 即可。

最后是几个比较简单的异常判断。比如向 `const` 类型变量赋值，我们只需要使用被赋值变量的 Token，根据 Token 的名称在符号表中查找到相应的 `Symbol`，检查 `Symbol` 的类型是否为 `const` 的几个类型即可。

这里还是要多加小心，在符号表中查找 `Symbol` 时，要按照作用域栈从栈顶到栈底的方向查找，确保在存在同名变量时优先查找到当前最内层的变量。

在检查 `printf()` 函数格式字符与表达式个数是否匹配时，我们向检查函数传入 `printf()` 函数的 Token，字符串常量，以及表达式的个数。我们使用正则表达式查找字符串常量中 `%d` 的个数，与表达式个数进行比对即可。

在检查 `continue` 和 `break` 语句是否在循环块中时，我们可以在 `Scope` 类对象中维护一个新的变量 `loopLayer`。在我们的文法中，只存在 `for` 循环一种循环。我们只需要在进入 `for` 循环时将 `loopLayer++`，退出 `for` 循环时将 `loopLayer--` 即可。在检查函数中，我们只需要判断当前 `loopLayer` 是否为 0，就可以完成对该异常的判断。

5.2 实现部分

【实现符号表管理】

为了实现符号表管理，我们首先创建枚举类 `SymbolType`，识别符号表中表项的种类：

```
// symbolizer/SymbolType
public enum SymbolType {
    ConstInt,
    ConstIntArray,
    StaticInt,
    .....
}
```

针对普通符号表和函数符号表，我们分别创建 `Symbol` 类和 `FuncSymbol` 类，用于存储符号表中的每个表项：

```
// symbolizer/Symbol
public class Symbol{
    private int scope;
    private String name;
    private SymbolType type;
}

// symbolizer/FuncSymbol
public class FuncSymbol {
    private int scope;
    private String name;
    private boolean hasReturnValue;
    private ArrayList<Boolean> params; // true: isArray, false: isNotArray
}
```

此外，我们还实现了 `Scope` 类，用于记录当前变量处在的作用域：

```
// symbolizer/Scope
public class Scope {
    private ArrayList<Integer> scopeStack;
    private int nowMaxScope;
    private int loopLayer;
}
```

接下来，我们实现了符号表管理类 `SymbolTable`，管理普通符号表和函数符号表在内的所有符号表，并进行异常检查：

```
// symbolizer/SymbolTable
public class SymbolTable {
    private Scope scope;
    private HashMap<Integer, ArrayList<Symbol>> symbols;
    private HashMap<Integer, ArrayList<FuncSymbol>> funcSymbols;
    private ErrorList errorList;
}
```

最后，我们实现了符号表管理的顶层类 `Symbolizer`，向其中传入语法分析得到的 AST，以及异常表 `errorList`，即可完成符号表构建的全流程：

```
// symbolizer/Symbolizer
public class Symbolizer {
    private final CompUnit compUnit;
    private Scope scope;
    private SymbolTable symbols;
    private ErrorList errorList;
}
```

符号表管理部分的文件结构如下：

```
src
|- symbolizer/
|   |- SymbolType
|   |- Symbol
|   |- FuncSymbol
|   |- Scope
|   |- SymbolTable
|   \- Symbolizer
|- ...
```

除此之外，我们还需要修改 AST 中的所有类，在其中加入 `symbolize()` 方法，实现符号表的构建，以及异常的检测。

6. 中间代码生成

6.1 设计部分

【常量赋初始值】

在中间代码生成部分中，我们需要保证常量能够在编译时被计算出初始值。为此，我们可以在 `expression` 中的所有类中加入 `calculate()` 方法，使用递归下降的方法计算表达式的值。

当然，如果表达式中只含有数字和运算符，那么计算过程应该是相当简单的。然而在实际上，表达式中还有可能会出现其它常量，例如下面的定义：

```
const int m = 5, n = 3;
const int a[m] = {m + n, m - n, m * n, m / n, m % n};
```

是的，你没看错，我们的公共测试库中竟然出现了形如 `a[m]` 这种的 VLA（这就是 VLA，即使 `m` 是 `const int` 它也是 VLA），让半夜 debug 的我喷出一口老血。

所以，为了能够在计算常量的值时，找到常量的值中的常量的值，我们需要在生成中间代码的同时，维护一个常量符号表，将已经计算出的常量和它们的值写入常量符号表中。

这样，每当我们在执行 `calculate()` 方法中遇到常量时，我们就可以根据常量的名称和当前的作用域，在常量符号表中寻找对应的初始值了。

【判断条件】

在 LLVM 中，`i1` 类型和 `i32` 类型之间有着严格的区别，我们使用 `icmp` 指令进行比较，得到的结果就是 `i1` 类型。

然而，我们的文法貌似是支持连续比较的，也就是形如 `5 > 3 > 2` 的格式。这个表达式的值看似为真，实则假，这是因为表达式首先执行了 `5 > 3`，返回的结果是表示真的 `1`，接下来执行 `1 > 2`，所以结果为假，返回的结果是表示假的 `0`。

有些跑题了，不过上面的例子在 LLVM 中确实也会出现一些问题，当我们在 LLVM 中执行 `5 > 3` 的时候，返回的其实是 `i1` 类型的 `1`，此时这个 `i1` 类型的 `1` 是不可以再与 `i32` 类型的 `2` 继续进行比较的，于是我们需要将其无符号扩展为 `32` 位。

为了防止上述情况，出于保守考虑，我们在所有 `icmp` 指令之后，都将 `icmp` 的结果使用 `zext` 指令无符号扩展为 `32` 位。

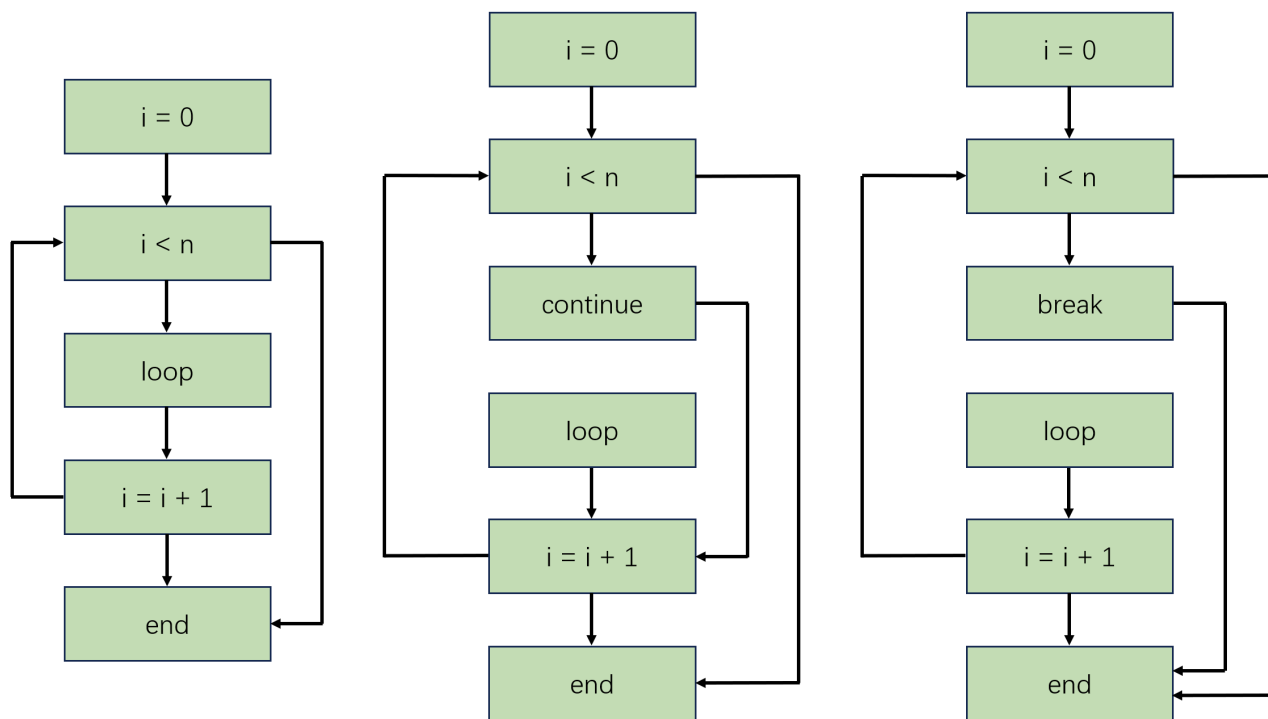
但是，在使用 `br` 进行条件跳转时，我们必须却又必须使用 `i1` 类型的变量进行条件判断。所以我们在所有条件跳转的 `br` 指令之前，都使用 `icmp` 指令比较判断条件是否和 0 不相等，从而将 `i32` 类型的判断条件再次转换回 `i1` 类型。

【分支与循环的基本块架构】

接下来我们来探讨一下分支与循环中的基本块架构。分支语句的基本块架构相对来说比较简单，其实就是当条件成立时，跳转到某一基本块，否则跳转到另一基本块，只需要一条 `br` 指令就可以轻而易举地实现。分支语句真正的难点在于它的判断条件，即之后的短路求值。

相比之下，`for` 循环的实现就略有些复杂了。虽然我们在计组课程中已经用 MIPS 写过很多次 `for` 循环了，但是这里是 LLVM，还是略有不同。在 LLVM 中，我们需要时刻注意，基本块的最后一条指令必须是 `br` 指令或 `ret` 指令；同时，`br` 指令和 `ret` 指令必须是基本块的最后一条语句！

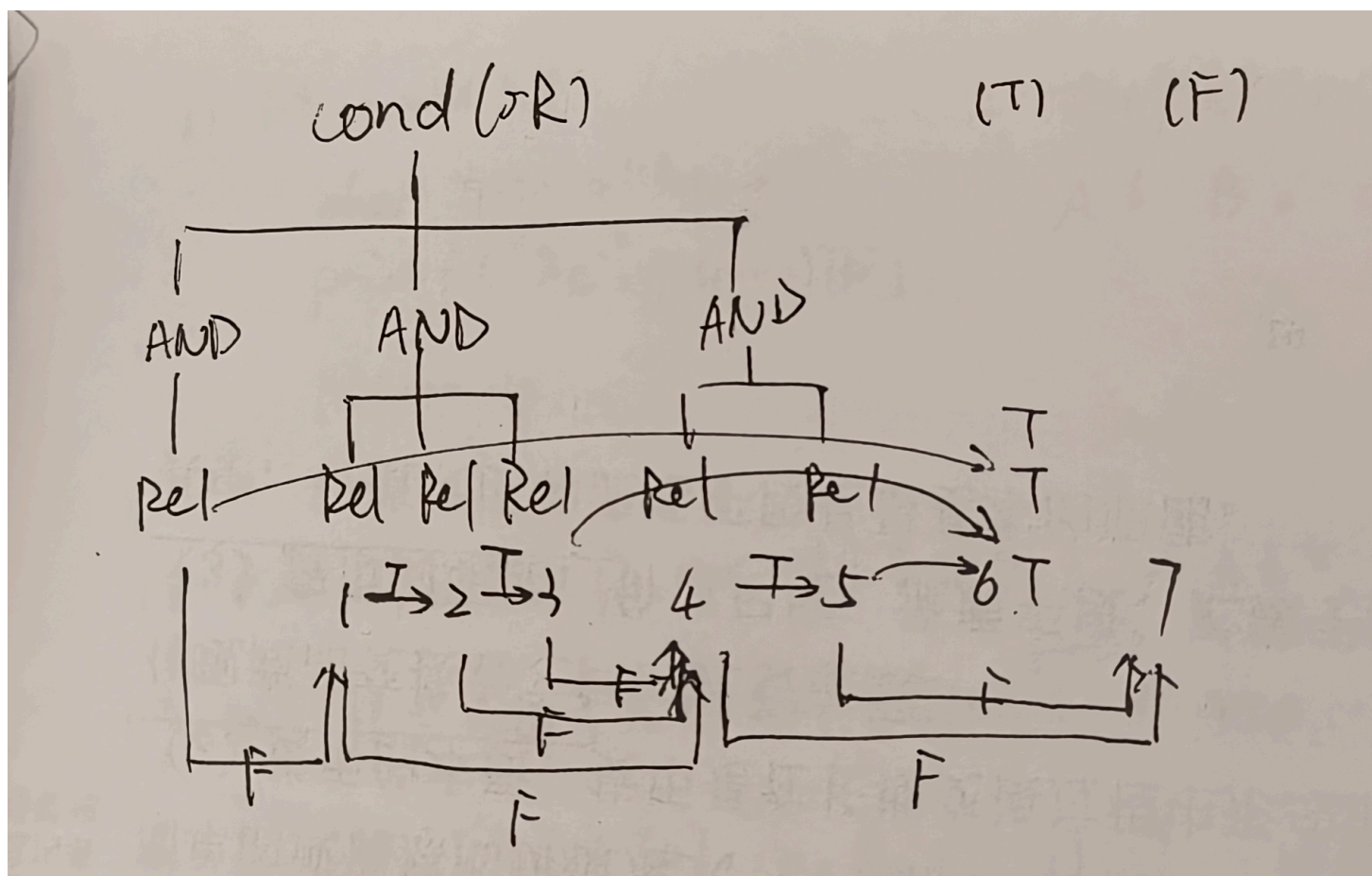
此外，和以往不同的时候，`for` 循环中可能会出现 `continue` 或者 `break`，此时基本块的架构就变得更加复杂了起来，具体架构可以查看下图：



【短路求值】

短路求值可以说是中间代码生成中最困难的一步。要实现短路求值，我们就不能简单地使用位运算代替 `&&` 和 `||` 运算，而是要将 `&&` 或 `||` 两侧的表达式视作基本块，进行精确的跳转操作。

从下面的（我手绘的）图中，我们能够观察到短路求值的跳转逻辑：



其中基本块 1 到 5 分别代表 5 个 RelExp 所在的基本块；基本块 6 表示整个条件为真时跳转到的基本块；基本块 7 表示整个条件为假时跳转的基本块。

通过观察可以发现，在同一个 LAndExp 中，若 RelExp 的值为真，如果当前 RelExp 不是 LAndExp 中的最后一个 RelExp，则跳转到同一个 LAndExp 中的下一个 RelExp；如果当前 RelExp 是 LAndExp 中的最后一个 RelExp，则直接跳转到条件为真时的基本块。

在同一个 LAndExp 中，若 RelExp 的值为假，如果当前 LAndExp 不是 LOrExp 中的最后一个 LAndExp，则跳转到下一个 LAndExp 所在的基本块；如果当前 LAndExp 是 LOrExp 中的最后一个 LAndExp，则直接跳转到条件为假时的基本块。

6.2 实现部分

【实现中间代码生成】

为了实现中间代码生成，我们创建 LLVM 接口，作为所有 LLVM 指令类的顶层接口。在 instruction 文件夹中，我们实现所有的 LLVM 指令类：

```
// llvmgenerator/LLVM
public interface LLVM {
    public void print(StringBuilder strb);
}
```

接下来，我们实现 LLVMTable 类，实现对所有 LLVM 指令的存储，以及最终 LLVM 指令的按顺序合并：

```
// llvmgenerator/LLVMTable
public class LLVMTable {
    private ArrayList<LLVM> llvms;
    private ArrayList<LLVM> llvmDefStrs;
    private ArrayList<LLVM> llvmDefStatics;
    private ArrayList<LLVM> mergedllvms;
}
```

最后，我们实现中间代码生成部分的顶层类 LLVMGenerator，将语法分析得到的 AST，以及语义分析中构建的符号表传入其中，即可得到生成后的中间代码：

```
// llvmgenerator/LLVMGenerator
public class LLVMGenerator {
    private final CompUnit compUnit;
    private SymbolTable symbols;
    private Scope scope;
    private LLVMTable llvms;
}
```

中间代码生成部分的文件结构如下：

```

src
|- llvmgenerator/
|   |- LLVMTable
|   |- LLVMGenerator
|   |- LLVM
|   \- instruction
|       |- LLVMAdd
|       |- LLVMAllocAdr
|       |- LLVMAllocVar
|       |- LLVMBranch
|       |- LLVMCall
|       |- LLVMDefFunc
|       |- LLVMDefFuncEnd
|       |- LLVMDefGlobalAdr
|       |- LLVMDefGlobalVar
|       |- LLVMDefStr
|       |- LLVMGetElementAdr
|       |- LLVMGetElementPtr
|       |- LLVMGetInt
|       |- LLVMIcmp
|       |- LLVMImport
|       |- LLVMJump
|       |- LLVMLabel
|       |- LLVMLoad
|       |- LLVMMul
|       |- LLVMPutInt
|       |- LLVMPutStr
|       |- LLVMRet
|       |- LLVMSdiv
|       |- LLVMSrem
|       |- LLVMStore
|       |- LLVMSub
|       \- LLVMZext
|- ...

```

除此之外，我们还需要修改 AST 中的所有类，在其中加入 `llvmGenerate()` 方法，实现中间代码的生成；我们还需要在符号表管理中加入 `ConstSymbol` 类，实现常量符号表的构建。

7. 目标代码生成

7.1 设计部分

【存储结构设计】

在 MIPS 中，不同于 LLVM 中无限量的虚拟寄存器，我们需要手动为每个变量分配存储位置。在这一部分中，我们暂时不实现寄存器分配，即不将变量存储在寄存器中，而是将所有变量全部存储在内存空间中。

目前采用的分配方法是：LLVM 中的局部变量（以 % 开头的变量）存储在栈中，以 \$sp 寄存器为基地址向下延伸；LLVM 中的全局变量（以 @ 开头的变量）和内存中的数据存储堆中，以 \$fp 寄存器为基地址向上延伸。

在使用一个变量时，我们需要将其 load 到临时寄存器中；在对一个变量赋值时，我们也需要将其 store 回内存中。这就涉及到一个问题，我们应该如何找到 LLVM 中的变量在 MIPS 内存中对应的位置？

对于 LLVM 局部变量，我们维护一个名为 spoffsets 的 HashMap，记录每个变量相对于 \$sp 寄存器的值的偏移量。例如，在程序第一次向 %3 赋值的时候，我们在 spoffsets 中以 %3 为 key 进行查找，如果没有找到，则将当前栈顶的位置减 4，将该位置写入 spoffsets 中，例如 -36。在下一次使用 %3 时，我们就能够在 spoffsets 中找到其对应的偏移量 -36，使用指令 lw \$t0, -36(\$sp) 即可将 %3 的值 load 到 \$t0 寄存器中；同样，在下一次对 %3 赋值时，我们也可以使用指令 sw \$t0, -36(\$sp)，就可以将 \$t0 寄存器的值写入栈中 %3 相应的位置了。

对于 LLVM 全局变量，我们在初始化的时候就可以将其写入 .data 段中，例如下面的语句：

```
.data
N: .word 10    #@N = dso_local global i32 10
a: .word 0, 1, 2, 3, 4, 5, 6, 7, 8, 9    #@a = dso_local global [10 x i32] [i32 0, i32 1, i32 2, i32 3, i32 4, i32 5, i32 6, i32 7, i32 8, i32 9]
str.0: .asciiz ", \0"    #@str.0 = private unnamed_addr constant [3 x i8] c", \0"
str.1: .asciiz "\n\0"    #@str.1 = private unnamed_addr constant [2 x i8] c"\n\0"
```

这里需要注意一些细节：MIPS 中的 label 不支持含有 @ 字符，我们在定义全局变量的时候需要删掉前面的 @ 字符；在 MIPS 中定义字符串时，我们需要将 \0A 和 \00 转换回 \n 和 \0；由于字符串的对齐问题，我们需要先定义全局变量，再定义字符串，否则会出现使用 lw 读取全局变量时未对齐的情况。

在使用 LLVM 全局变量的时候，我们可以使用 la 伪指令，即可将全局变量的地址写入寄存器中，例如上述例子中，使用 la \$t0, N 即可把 N 的地址写入 \$t0 寄存器中。在这之后，我们就可以配合 LLVM 中的 load 指令获取全局变量的内容了。

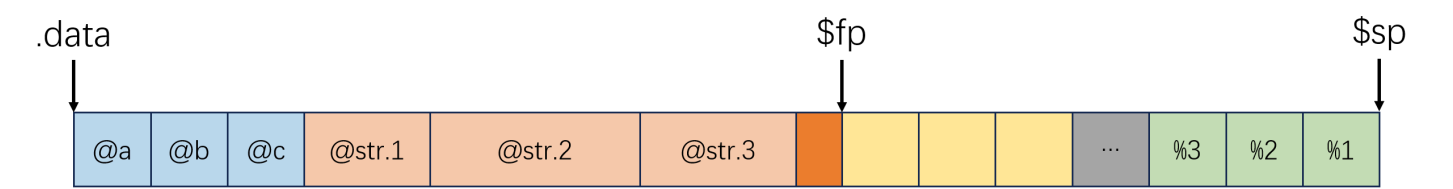
对于 LLVM 中存储于内存中的内容，即 LLVM 中的 `alloca` 指令分配的空间，我们将其存储于堆中全局变量的后面。

然而，这样实现可能会出现一个问题，那就是我们在翻译 `alloca` 指令时，该如何确定该指令写入寄存器的值，即我们分配的内存的地址。当然，我们其实也可以手动计算出所有全局变量所占用的空间，作为内存空间的基地址，但这样还是太麻烦了！得益于 MIPS 中灵活的操作技巧，我们可以在所有的全局变量后面自动添加一个 `label align`，在 `.text` 段的最开始固定添加一条指令 `la $fp, align`，这样就把全局变量的末尾地址，也就是内存空间的基地址写入了 `$fp` 寄存器中，我们只要将分配的内存空间相对于 `$fp` 的位置 `offset` 作为 `alloca` 指令写入寄存器的值，即可在使用内存空间的时候使用 `lw $t0, offset($fp)` 获取到其中的内容了。

另外，我们还是需要注意 MIPS 内存的字对齐问题。如果我们直接将内存空间置于全局变量的末尾，因为字符串的存在，就可能会出现字不对齐的情况。恰巧 MIPS 中有这样一个东西，使用 `align: .align 2`，即可让 `align` 标签的位置自动实现字对齐。于是我们生成的指令结构如下：

```
.data
N: .word 10    #@N = dso_local global i32 10
a: .word 0, 1, 2, 3, 4, 5, 6, 7, 8, 9    #@a = dso_local global [10 x i32] [i32 0, i32 1, i32 2, i32 3, i32 4, i32 5, i32 6, i32 7, i32 8, i32 9]
str.0: .asciiz ", \0"    #@str.0 = private unnamed_addr constant [3 x i8] c", \00"
str.1: .asciiz "\n\0"    #@str.1 = private unnamed_addr constant [2 x i8] c"\0A\00"
align: .align 2

.text
la $fp, align
```



【函数设计】

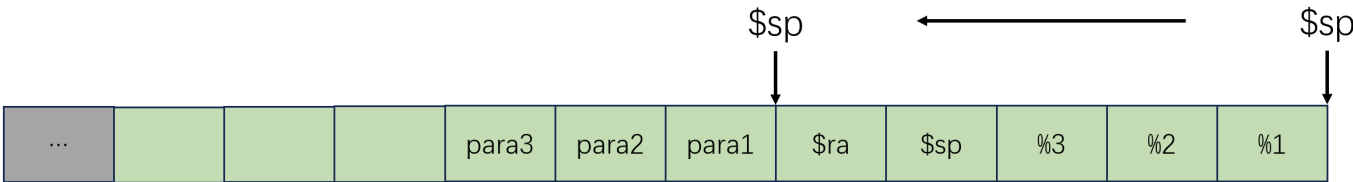
不同于 LLVM 近似于 C 语言的函数格式，在 MIPS 中，我们需要自行完成函数的跳转和返回、参数和返回值的传递、寄存器的压栈和弹出、栈空间的切换，这些都是需要我们攻坚克难的大问题。

首先是函数的跳转和返回，这个我们都比较熟悉了，在计组课程中已经练得炉火纯青，使用 `jal` 指令跳转到目标函数，再使用 `jr` 指令即可返回原函数。

接下来是参数和返回值的传递。返回值的传递比较简单，因为一个函数最多有一个返回值，所以我们只需要按照国际惯例，将返回值存储在 \$v0 寄存器中即可。然而函数参数的个数是无限的，即使使用常见的 \$a0 - \$a3 寄存器存储参数，函数参数超过 4 个的情况依旧比比皆是。所以我们直接放弃使用寄存器存储参数，直接将函数的所有参数全部压入栈中。这里需要注意的是，这一步必位于寄存器的压栈之后，目的是在栈空间切换的时候，压入的参数能够位于栈空间的顶部。

其次就是寄存器的压栈和弹出，这一部分我们也是非常熟了，由于我们并没有使用 MIPS 寄存器存储任何变量的值，所以我们只需要将 \$sp 和 \$ra 两个寄存器压入栈中即可。在函数返回时，我们也只需要将这两个寄存器进行弹出。

最后是栈空间的切换。我们在进入函数内部时，需要通过将 \$sp 寄存器的值移动到栈顶的方式开辟出一片新的栈空间；在函数返回时，再将 \$sp 寄存器的值移动回之前的位置。然而，为了参数的传递，我们不能直接将 \$sp 寄存器的值减去当前栈顶的偏移量，而是应该再加上函数参数的个数乘 4 的值，即移动到栈空间中函数参数的底部。这样一来，神奇的事情就发生了：我们之前压入栈中的参数值，恰好位于新栈空间的底部。此时我们依次为 LLVM 变量 %0, %1, %2... 分配栈空间，就恰好与压入栈中的参数值对应！如下图所示：



【label设计】

7.2 实现部分

【实现目标代码生成】

为了实现目标代码生成，我们首先创建枚举类 Register，列出 MIPS 中所有的寄存器，便于后续调用和分配：

```
// mipsgenerator/Register
public enum Register {
    $zero,
    $at,
    $v0,
    .....
}
```

接下来，我们创建 MIPS 接口，作为所有 MIPS 指令类的顶层接口。在 instruction 文件夹中，我们实现所有的 MIPS 指令类：

```
// mipsgenerator/MIPS
public interface MIPS {
    public void print(StringBuilder strb);
}
```

在此之上，我们创建 MIPSTable 类，作为所有 MIPS 指令的集合，存储所有 LLVM label 在堆栈中的位置，同时预留分配寄存器的接口：

```
// mipsgenerator/MIPSTable
public class MIPSTable {
    private ArrayList<MIPS> mipsDataSegments;
    private ArrayList<MIPS> mipsTextSegments;
    private HashMap<String, Register> regs;
    private int nowfpoffset;
    private HashMap<String, Integer> spoffsets;
    private int nowspoffset;
    private String nowFunc;
    private int callCount;
    private ArrayList<Register> pool;
    private int poolindex;
}
```

最后是目标代码生成部分的顶层类 MIPSGenerator 类，我们向其中传入 LLVM 指令集合，即可得到翻译之后的 MIPS 指令集合：

```
// mipsgenerator/MIPSGenerator
public class MIPSGenerator {
    private final LLVMTable llvms;
    private MIPSTable mipses;
}
```

目标代码生成部分的文件结构如下：


```

src
|- mipsgenerator/
|   |- Register
|   |- MIPSTable
|   |- MIPSGenerator
|   |- MIPS
|   \- instruction
|       |- MIPSAdd
|       |- MIPSAddi
|       |- MIPSBranchIfEqualZero
|       |- MIPSBranchIfNotEqualZero
|       |- MIPSDefGlobalArr
|       |- MIPSDefGlobalVar
|       |- MIPSDefStr
|       |- MIPSGetInt
|       |- MIPSJump
|       |- MIPSJumpAndLink
|       |- MIPSJumpRegister
|       |- MIPSLabel
|       |- MIPSLoadAddr
|       |- MIPSLoadImm
|       |- MIPSLoadWord
|       |- MIPSMove
|       |- MIPSMul
|       |- MIPSOver
|       |- MIPSPutInt
|       |- MIPSPutStr
|       |- MIPSsdiv
|       |- MIPSSet
|       |- MIPSShiftLeftLogical
|       |- MIPSShiftRightArithmetic
|       |- MIPSsrem
|       |- MIPSStoreWord
|       \- MIPSSub
|- ...

```

除此之外，我们还需要修改全部的 LLVM 指令类，在其中加入 `mipsGenerate()` 方法，实现向 MIPS 指令的翻译。

8. 代码优化

8.1 设计部分

代码优化的设计部分在优化文章中有非常详细的介绍，在这里就仅简单介绍一下已实现的优化和优化的思路。

【针对MIPS的结构优化】

我们在使用 LLVM 作为中间代码的时候，受限于 LLVM 语法的严格限制，可能会需要进行一些非常复杂的操作，如频繁读写内存，将指向数组的指针转换为普通指针，i1 和 i32 类型之间互相转换等等。而这些操作是我们在 MIPS 的目标代码中完全不需要的。所以，为了能够减少无用的操作，我们可以针对 MIPS 的需求优化中间代码的形式，或者对生成的目标代码进行微调。

首先我们可以优化中间代码中变量的存储形式。对于 LLVM 中的非数组局部变量，我们不再为其分配空间，而是直接将值存入 LLVM 的寄存器中；对于 LLVM 中的数组，我们在分配空间后不再返回指向数组的指针，而是返回指向数组首地址的指针。

在 MIPS 中，我们申请任意一片空间，空间内的所有值默认均为 0。于是在生成中间代码的过程中，如果数组局部变量的某个元素值为 0，那么我们就不需要计算它的指针位置，也不需要给它赋值为 0 了。

在 LLVM 中，i1 和 i32 类型有着严格的区别，不能混在一起运算，而 MIPS 中所有的值类型均为 32 位，不存在这一问题。所以我们可以删去生成中间代码中为了保证类型正确进行的保守操作，如使用 zext 将 i1 类型转换为 i32 类型，以及使用 icmp sne 将 i32 类型转换为 i1 类型的语句。

在 LLVM 中，函数的任何一个基本块都必须以 br 指令或 ret 指令结尾，而 MIPS 中则不然。在 MIPS 中，如果 label 的前一条指令不是跳转指令，程序运行时会直接步入 label 中。于是我们可以在生成目标代码时，检查当前的跳转是否为直接步入了下一条语句，如果是的话，则可以删除当前语句。

【全局寄存器分配】

全局寄存器分配包括划分基本块、活跃变量分析、构建冲突图、图着色分配寄存器等多个环节，在这里进行简单的介绍。

在进行目标代码生成时，我们每当进入一个新的函数，就为函数中的所有 LLVM label 进行全局寄存器分配。首先是进行基本块的划分。我们根据 LLVM label 的位置将函数划分为多个基本块，接下来根据每个基本块末尾的 br 指令或 ret 指令确定基本块之间的执行顺序，构建基本块的流图。

接下来，我们需要进行活跃变量分析。我们首先正序遍历每个基本块内部的每条指令，求出每个基本块的 def 集合和 use 集合。接下来，我们倒序遍历每个基本块，求出每个基本块的 in 集合和 out 集合。我们不断重复倒序遍历基本块的过程，直到所有基本块的 out 集合均不再发生变化为止。

在进行了活跃变量分析之后，我们要根据每个基本块的 `out` 集合构建冲突图。这里我们认为两个变量冲突的定义是，在一个变量的定义点处，另外一个变量活跃。于是在每个基本块内部，我们倒序遍历其中的每条指令，计算每个定义点处的活跃变量集合，这样就能够成功构建出冲突图。

获得冲突图后，我们还需要为冲突图进行图着色，并分配全局寄存器。我们遍历冲突图中的所有节点，获取度最小的节点和度最大的节点。如果最小的度小于能够分配的寄存器数量，那么我们就为度最小的节点分配全局寄存器，并将其从冲突图中删除；反之，则直接将度最大的节点从冲突图中删除，不为其分配全局寄存器。

我们不断重复上述操作，直到冲突图中不再含有节点。这样，我们就完成了全局寄存器分配。

【其它简单的优化】

在前两个比较大的优化的基础上，还有一些比较简单的优化值得探索，例如常量折叠。在生成中间代码的时候，若当前指令是计算类指令，例如 `add` `sub` 指令，若进行计算的两个操作数均为常数，我们就可以直接对其进行运算，返回运算的结果即可，而不需要生成一条对应的指令。

8.2 实现部分

【全局寄存器分配的实现】

上面提到的优化，基本上都是在代码生成的过程中缝缝补补，并没有进行比较大范围的修改。而全局寄存器分配的代码量较大，这里采用了一个新的类 `Optimizer` 来实现这个功能：

```
// mipsgenerator/Optimizer
public class Optimizer {
    private ArrayList<LLVM> llvms;
    private ArrayList<Register> pool;
    private ArrayList<String> blockNames;
    private HashMap<String, ArrayList<LLVM>> blocks;
    private HashMap<String, HashSet<String>> in2out;
    private HashMap<String, HashSet<String>> out2in;
    private HashMap<String, HashSet<String>> def;
    private HashMap<String, HashSet<String>> use;
    private HashMap<String, HashSet<String>> in;
    private HashMap<String, HashSet<String>> out;
    private HashMap<String, HashSet<String>> crushMap;
    private HashMap<String, Register> label2regs;
    private HashMap<LLVM, ArrayList<String>> activeLabelsWhenCall;
}
```

其具体的功能在优化文章中有详细的说明，这里就不再赘述了。